

Taller 02 — RAG sobre un corpus real: baseline medible, abstención y una extensión

Materia: MMIA 6013 IA Generativa y Agentes **Semana:** 2 · **Sesión:** 10 (sábado, 230 min) **Peso:** 25 % de la nota final
Fecha de entrega: 26 de septiembre de 2026 — el repositorio con `resultados.csv` al cierre de la sesión (11:50); el informe en PDF, con la Parte 4, **hasta las 23:59 del mismo día**, por el canal que se indique en clase **Modalidad:** grupos de 2–3 personas

Contexto

Esta semana se recorrió el camino completo: qué es un embedding y qué cambia la normalización (lunes), dónde vive un vector y cómo se encuentra sin mirarlos todos (martes), cómo se fragmenta un documento y qué recupera cada tipo de búsqueda (miércoles), y cómo se mide si la recuperación funciona con un golden set que tenga dueño, versión y preguntas sin respuesta (jueves). Este taller construye **un RAG baseline reproducible sobre un corpus real**, lo mide, y le añade **una** extensión elegida por el modo de falla que el baseline exhibió, no por preferencia.

La regla de oro del taller es la del miércoles: **baseline → medir → extender → medir**. Un RAG simple con golden set y métricas honestas puntúa mejor que un sistema complejo que «parece» funcionar. Y las métricas honestas empiezan por saber qué **no** miden: por eso hay una Parte 0 antes de tocar tu corpus.

Requisito del corpus. Mínimo 5 documentos y 50 páginas en total, en un solo idioma que puedas declarar, y con **capa de texto**: un PDF escaneado no aporta nada a un índice (lo vas a comprobar en la Parte 0.a). Si tu corpus trae uno, lo detectas, lo declaras y decides qué hacer con él; eso también se califica.

Con qué modelo corre cada parte

El profesor reparte en clase una **clave de OpenAI**, y la **tabla semestral de modelos** del curso tiene desde el 27 de agosto una clave `embeddings` y desde hoy una clave `rerankers`. Los embeddings se calculan con **bge-m3 en la H200 de la USFQ**, a costo cero y con la VPN GlobalProtect conectada; **sin VPN, la alternativa es la API de OpenAI**. Cada parte declara con qué corre; la generación también corre en local.

Parte	Con la clave de OpenAI	Sin clave, a costo cero
0	— (no usa API)	<code>embed_notebook_s2</code> en tu máquina (<code>EMBEDDING_BACKEND=local</code>) — obligatoria , y la única parte que no usa <code>bge-m3</code>
1.3 embeddings	<code>embed_api_economico</code> (8192 tokens), solo si no tienes VPN	<code>embed_local_multilingue</code> — <code>bge-m3</code> — en la H200 de la USFQ (8192 tokens, VPN GlobalProtect): el baseline de todos
1.5 generación	<code>propietario_economico</code>	<code>open_weight_pequeno</code> por Ollama
2.b Hit Rate y MRR	— (no usa API)	el evaluador, en local
2.b tasas de abstención	el mismo LLM de la 1.5	el mismo local
3.C reranking	—	<code>rerank_local_ingles</code> o <code>rerank_local_multilingue</code> , según el idioma
3.D RAGAS	<code>propietario_economico</code> como juez	sin ruta local declarada: se dice

Cuatro cosas que esta matriz decide por ti:

1. **El modelo de embeddings es `bge-m3`, y es multilingüe**: sirve para las tres opciones de corpus. Se declara por su `id` — `embed_local_multilingue` — con su `verified_at`. Si no tienes VPN y usas OpenAI, declaras esa fila; y un índice construido con un modelo **solo se consulta con ese mismo modelo**: si cambias de ruta, reindexas. Un modelo monolingüe inglés sobre un corpus en español recuperaría mal y **el índice no se quejaría**: devolvería sus k vecinos con distancias de aspecto razonable y ninguno respondería. Se parece a un mal corte de fragmentos y no lo es; distinguirlo es media Parte 2.c.
2. **`bge-m3` admite 8192 tokens de secuencia; el MiniLM de la Parte 0, 128**. El `Lab-02` fragmenta en **tokens del mismo modelo**, lee el tope en vez de suponerlo, y por defecto corta en 512 con un quinto de solapamiento. La Parte 0.b usa el MiniLM precisamente porque trunca: es donde se ve lo que un tope le hace a un fragmento.
3. **La Parte 2 no gasta nada**: Hit Rate y MRR salen del índice. Solo las dos tasas de abstención generan, y generan una vez por pregunta. No hay razón para no correrla entera.
4. **El reranker de la Opción C está entrenado en inglés** (MS MARCO). La variante multilingüe existe y no publica cifras: que sea mejor sobre **tu** corpus se mide, no se supone. Es lo que la Opción C pide reportar.

Corpus disponible

Elige **uno** (o propón otro con aprobación del profesor):

Opción A — Documentos institucionales USFQ, en español: Código de honor y convivencia, Instructivo de Prácticas de Campo, Instructivo de Graduación, Reglamento académico (si está disponible).

Opción B — Papers del curso, en inglés: los papers seminales de las semanas 1 y 2 (los PDF están en el repositorio del curso).

Opción C — Corpus propio: documentos de tu dominio (regulaciones, manuales técnicos, papers de tu área). Mínimo 5 documentos, mínimo 50 páginas, un solo idioma declarado.

Parte 0 — Tres fallas que no fallan (10 %)

Objetivo

Antes de indexar tu corpus, ver con tus propios ojos tres cosas que un pipeline RAG hace en silencio si nadie lo comprueba, sobre el andamiaje tal como viene y sobre `Lab-02/ejemplos/`. Sin clave, sin VPN y sin Docker: `EMBEDDING_BACKEND=local QDRANT_URL=":memory:"`, en diez minutos. Cada una es la evidencia que la Parte 2.c te va a pedir para diagnosticar tu corpus.

Si alguna de las tres no se ve, cambiaste algo del andamiaje y tienes que decir qué. El `Lab-02` detecta las tres y lo dice en pantalla; la Parte 0 te pide explicar qué pasaría si no lo hiciera.

o.a — El PDF que se indexa vacío (3 %)

`ejemplos/instructivo_escaneado.pdf` no tiene capa de texto. Ingesta la carpeta `ejemplos/` con `python rag_pipeline.py` y pega la salida de la ingesta. Después, en `ingestion.py`, explica en dos frases qué comprueba `load_corpus` para no indexarlo y **qué produciría sin esa comprobación** con ese mismo archivo: un fragmento hecho de `[page=1]` `[page=2]`, sin una palabra del documento y sin ninguna excepción.

Entregable 0.a: la salida cruda de la ingesta y las dos frases. Si tu corpus real trae un documento así, va aquí también.

o.b — El fragmento que se corta a 128 (4 %)

Con el MiniLM de la Parte 0 (`EMBEDDING_BACKEND=local`, tope de 128 tokens), ingesta `ejemplos/` pidiendo `chunk_tokens=900` y pega el aviso. Después toma un texto de unas 900 palabras, cuenta sus tokens con el tokenizador del modelo, y compara el vector del texto entero con el del texto recortado a `max_seq_length` tokens: el coseno es ≈ 1 . **Todo lo que no cupo nunca llegó al índice.** Es el ejercicio 5.1 del lunes, sobre el laboratorio.

Entregable 0.b: el aviso, los dos números (tokens del texto y tope del modelo), el coseno, y **el `chunk_tokens` que vas a usar en la Parte 1 con una frase que lo justifique** contra el tope de `bge-m3`: que 8192 quepan no quiere decir que un fragmento de 8192 sea bueno.

o.c — El índice que no se queja (3 %)

Sobre `ejemplos/` indexado, pregunta algo cuya respuesta no está ("`¿cuál es la política de mascotas?`") y pega los tres vecinos con sus puntajes. Tienen aspecto sano. Después pregunta algo cuya respuesta **sí** está en el PDF escaneado que la 0.a no indexó ("`¿qué exige el instructivo de prácticas de campo sobre el seguro de accidentes?`") y pega los tres vecinos. También tienen aspecto sano.

Entregable 0.c: las dos salidas y una frase sobre por qué ningún Hit Rate puede detectar esto, que es la razón de las preguntas negativas de la Parte 2.a.

Parte 1 — Baseline RAG funcional (35 %)

Instrucciones

Implementa el pipeline sobre tu corpus, a partir del Lab-02:

1. **Ingesta.** Parsea los documentos (PDF → texto). Documenta qué se pierde o se degrada (tablas, encabezados, pies de página) y qué documentos, si alguno, la ingesta rechazó por no tener texto útil.
2. **Fragmentación.** Baseline de tamaño fijo con solapamiento, **en tokens del modelo de embeddings**, con `chunk_tokens` y `overlap_tokens` explícitos. El tope de `bge-m3` es 8192 y el Lab-02 corta por defecto en 512: la justificación dice por qué tu tamaño sirve a tus preguntas, no solo que cabe.
3. **Embedding e indexación.** Con `bge-m3` en la H200 (`EMBEDDING_BACKEND=h200`, el valor por defecto, con GlobalProtect conectada) o, sin VPN, con OpenAI (`EMBEDDING_BACKEND=openai`). Declara en el informe la fila por su `id` y con su `verified_at`. Indexa en Qdrant (el contenedor, no `:memory:`, que es solo para depurar).
4. **Recuperación.** Muestra los top-5 fragmentos para al menos 3 preguntas de prueba, con documento y puntaje.
5. **Generación.** `rag_pipeline.generate` ya trae la llamada: con la clave del curso usa la fila `propietario_economico`; sin clave, Ollama con `open_weight_pequeno`. El prompt del baseline hace tres cosas —delimita el contexto, obliga a usar solo el contexto y **autoriza la abstención con una frase fija**— y esa frase es la misma en el prompt, en el golden set y en el detector: `ABSTENCION` en `rag_pipeline.py`. No la cambies sin cambiarla en los tres sitios.

Entregables

Código reproducible; los parámetros de fragmentación con su justificación; la fila de embeddings con su `id` y fecha; las tres preguntas de prueba con sus top-5.

Parte 2 — Evaluación con golden set (30 %)

2.a — El golden set es el verificador (10 %)

Construye **10 pares** pregunta-respuesta esperada-fuente sobre tu corpus, en `golden_set.json` a partir de `golden_set_plantilla.json`:

- preguntas **simples** (la respuesta está en un fragmento), **multi-fragmento** (hay que combinar partes del corpus), y **al menos 2 negativas** (la respuesta no está en el corpus);
- **anotadas por documento fuente y, cuando puedas, por una frase literal del documento** (`documentos_fuente`, `fragmento_esperado`), **no por identificador de fragmento**: los `chunk_id` cambian al re-fragmentar y una anotación por identificador se invalida sin fallar en cuanto pruebes otra estrategia en la Parte 3;
- la **adversarial** (una instrucción metida en la pregunta) es responsable si le das documento fuente; si prefieres que el sistema se abstenga, déjala sin fuente y contará como negativa. Decide y dilo.

Como en el Taller 1: **el verificador es un entregable y su diseño se califica**. Diez preguntas con las palabras exactas del documento dan Hit Rate 1,0 y cero información.

2.b — Las métricas, sobre lo que cada una puede medir (10 %)

`python evaluation.py --k 3` y `--k 5`. Reporta:

- **Hit Rate@k** y **MRR** sobre las preguntas **responsibles**. MRR es la **media de los recíprocos** de la posición del primer acierto, no la posición promedio: no son la misma cifra ni se mueven igual.
- Las **dos tasas de abstención**: la **correcta** (negativas en las que el sistema se abstuvo) y la **indebida** (responsibles en las que también se abstuvo). Son dos números, no uno: un sistema que siempre se abstiene tiene la primera en 1,0 y es inútil.

Las negativas **no entran** en Hit Rate ni en MRR: no hay nada que recuperar. Si entraran, con tres negativas de diez un sistema perfecto reportaría 0,70 como techo.

2.c — Análisis de fallos (10 %)

Los **tres peores casos**, y para cada uno la parte del pipeline a la que apunta —ingesta, fragmentación, recuperación, prompt o generación— **con evidencia**: la salida de la ingesta para ese documento, los fragmentos recuperados con sus puntajes, la respuesta generada. La Parte 0 te enseñó la forma de cada una de las tres fallas silenciosas; aquí tienes que reconocerlas en tu corpus, o decir con evidencia que no están.

Extensión recomendada

Si el baseline ya funciona, calcula **RAGAS Faithfulness** y **Answer Relevancy** (Opción D, abajo).

Parte 3 — Extensión elegida por el modo de falla (10 %)

Elige **una, por el par (Hit Rate, MRR) y las tasas de abstención que mediste**, y declara antes de implementarla qué modo de falla esperas que corrija. El verbo es **mide y explica**, no «mejora»: las lecturas obligatorias no prometen ganancia numérica del híbrido ni del reranking, y una extensión que no mejora sobre tu corpus es un resultado válido si explicas por qué.

Opción A — Segunda estrategia de fragmentación. Semántica, jerárquica o contextual. Compara Hit Rate y MRR contra el baseline **con el mismo golden set**: por eso se anota por documento y frase, y no por `chunk_id`.

Opción B — Búsqueda híbrida. BM25 sobre el corpus (`rank_bm25` o la función nativa de Qdrant), fusionado con el denso por *Reciprocal Rank Fusion* (RRF). Mide Hit Rate y MRR contra el baseline; el jueves implementaste RRF sobre el corpus de ejemplo.

Opción C — Reranking. Recupera top-20 con el modelo de embeddings, reordena con un cross-encoder, entrega top-5. Las dos filas de la clave `rerankers` de la tabla semestral: `rerank_local_ingles` (MS MARCO, inglés, con cifras publicadas) y `rerank_local_multilingue` (mMARCO, 14 idiomas, **sin cifras publicadas**). Elige por idioma, declárala por su `id`, y mide el MRR contra el baseline.

Opción D — RAGAS. Faithfulness y Answer Relevancy sobre el golden set, con `propietario_economico` como juez. **Estima el costo antes** (Presupuesto, abajo), reporta costo y número de llamadas, las limitaciones del juez, y una muestra de casos donde el puntaje no coincide con tu juicio.

Parte 4 — Reflexión (5 %)

Responde brevemente (100–150 palabras por pregunta):

- 1. **Con tu tabla de la Parte 2.b delante.** Hasta la revisión anterior del evaluador, un sistema que acertaba **todas** las preguntas respondibles reportaba un Hit Rate de 0,70 con la plantilla tal como se distribuía. ¿Qué media ese número, qué cambia al sacar las negativas del denominador, y por qué la métrica correcta para ellas es otra? Responde con tus cifras, no de memoria.
- 2. **Una pregunta global.** Edge et al. (2024) distinguen las preguntas que un fragmento puede contestar de las que exigen resumir el corpus entero (§1). Escribe una pregunta **global** sobre tu corpus, explica por qué tu RAG vectorial no puede contestarla bien, y qué haría GraphRAG distinto con ella.

Presupuesto — léelo antes de generar

Solo dos partes gastan clave, y las dos generan una vez por pregunta:

Parte	Dónde corre	Llamadas
0, 1.1–1.4, 2.a, 2.b (Hit Rate, MRR), 3.A, 3.B, 3.C	local	0
1.5 generación y 2.b abstención	propietario_economico	10 preguntas × 2 valores de k
3.D RAGAS	propietario_economico como juez	del orden de decenas por pregunta : estímalo
todo, sin clave	open_weight_pequeno por Ollama	0 USD

Con los precios de la fila y su fecha (anexo), las Partes 1 y 2 cuestan **céntimos** por grupo. La única parte que puede sorprender es RAGAS: **corre una pregunta completa antes de correr las diez**, cuenta las llamadas, y multiplica. La estimación va en el informe, con la fila y su fecha; una cifra sin fecha de verificación pierde los puntos de reproducibilidad.

Entregables

- **Informe en PDF:** las tres salidas de la Parte 0; la arquitectura; los parámetros de fragmentación en tokens con su justificación; la fila de embeddings con `id` y fecha; la tabla de métricas de la Parte 2.b (Hit Rate@3 y @5, MRR, las dos tasas); los tres peores casos con su evidencia; la extensión con su medición; las respuestas de la Parte 4.
- **Repositorio:** código reproducible, `requirements.txt` con versiones, `golden_set.json`.
- **resultados.csv crudo**, el que escribe `evaluation.py`: una fila por consulta, con `id`, tipo, modelo de embeddings, k, posición del primer acierto, acierto, recíproco, puntaje del mejor vecino, abstención y generador. **Las tablas del informe se derivan de ese archivo; sin él no hay reproducibilidad que calificar.**

Ninguna clave de API puede aparecer en el informe, en el notebook, en una captura ni en el repositorio (política de credenciales del curso). Es un hallazgo bloqueante, no una advertencia: se leen de un `.env` que el `.gitignore` excluye.

Criterios de evaluación

Criterio	Peso
Parte 0 — Las tres fallas reproducidas, con salida cruda y la explicación de qué las detecta	10 %
Parte 1 — Baseline funcional, con fragmentación en tokens justificada y fila de embeddings declarada con fecha	35 %
Parte 2.a — Golden set de diez, tres tipos, dos negativas, anotado por documento y frase	10 %
Parte 2.b — Hit Rate y MRR sobre responsables, las dos tasas de abstención, para dos valores de k	10 %
Parte 2.c — Tres peores casos con la parte del pipeline señalada y evidencia	10 %
Parte 3 — Extensión elegida por el modo de falla medido, medida y explicada	10 %
Parte 4 — Reflexión	5 %
Reproducibilidad: repositorio, versiones, <code>resultados.csv</code> crudo y ausencia de credenciales	10 %

Los encabezados de parte llevan **estos mismos números**. Terminado no es «el retrieval anda»: un informe con métricas pero sin la fila de embeddings fechada, o con una tabla que no se puede reconstruir desde el CSV, pierde los puntos de esas filas aunque el código sea correcto.

Anexo — la tabla semestral de modelos

Este enunciado cita las filas **por su id** —«la fila `embed_local_multilingue`», «la fila `rerank_local_multilingue`»— y quien trabaje con el PDF impreso no tiene el archivo delante. Aquí están las que importan hoy.

Las tablas de abajo se generan desde la tabla semestral de modelos del curso cada vez que se construye este PDF.**** No están escritas a mano y no pueden desincronizarse del archivo: si un tope de tokens cambia, se regenera el PDF y cambia aquí. **La fila que uses en tu informe va con su id y con la fecha de esa fila.**

Modelos de embeddings

📄	proveedor	modelo	dimensión	tokens de secuencia	multilingüe	USD/1M	verificado
embed _local_ _rapid o	sentence- transformer s	sentence- transformers/all- MiniLM-L6-v2	384	128	no	0 · cero real	2026-08- 27
embed _noteb ook_s 2	sentence- transformer s	sentence- transformers/para phrase- multilingual- MiniLM-L12-v2	384	128	sí	0 · cero real	2026-08- 27
embed _local _multi lingu e	BAAI	BAAI/bge-m3	1024	8192	sí	0 · cero real	2026-08- 27
embed _api_e conomi co	openai	text-embedding-3- small	1536	8192	sí	0,02	2026-08- 27
embed _api_g rande	openai	text-embedding-3- large	3072	8192	sí	0,13	2026-08- 27

Reordenadores (cross-encoders) — la Opción C

📄	modelo	entrenado en	multilingüe	cifra publicada en la <i>model card</i>	verificado
rerank _local_ ingles	cross- encoder/ms- marco-MiniLM- L6-v2	MS MARCO Passage Ranking (ingles)	no	ndcg at 10 trec dl 2019 = 74.3 · mrr at 10 msmarco dev = 39.01	2026-09- 19
rerank _local_ multili ngue	cross- encoder/mmarco- mMiniLMv2-L12- H384-v1	mMARCO: MS MARCO traducido automaticamente a 14 idiomas (Google Translate)	sí	ninguna	2026-09- 19

Cómo se lee

- **Tokens de secuencia** es el tope del modelo: con el MiniLM, un fragmento más largo se vectoriza truncado **sin avisar** en la librería, y el **Lab-02** es quien avisa (Parte 0.b); contra la H200, el **Lab-02** pide que un texto que no cabe dé error. Es la cifra que acota la Parte 1.2.
- **Multilingüe** decide la fila por el idioma del corpus, no por preferencia. Una fila monolingüe sobre otro idioma no falla: recupera mal en silencio.

- Las cifras de los reordenadores son las que publica cada *model card*; **ninguna es una medición del curso**, y la fila multilingüe no publica ninguna. Sobre tu corpus, se mide.